B6 Reproducibility

Seeds, config, data versioning and pinned dependencies.

B6-L1 Seeds and random generators

Seed once at the edge, pass the generator in, and never use hash() for anything that must be stable.

```
rng = np.random.default_rng(20260925)      # at the edge, once
sample = draw_sample(ids, k=500, rng=rng)  # the function receives rng
```

Reproducible randomness in NumPy 2	rng = np.random.default_rng(seed), created once at the edge and passed in.
Why not np.random.seed?	It seeds one global generator that every library shares and advances.
Stable bucket for an ID	int(hashlib.sha256(id.encode()).hexdigest(), 16) % n. Never hash(), which is randomised per process.
Independent streams for workers	rng.spawn(n).

B6-L2 Record the data, config and code behind every result

Hash the input and the loaded config, record the commit and the seed, and store the manifest with the output.

```
manifest = {
    "run_id": run_id,
    "started_utc": datetime.now(timezone.utc).isoformat(),
    "input_sha256": file_sha256(input_path),
    "config_sha256": config_sha256(config),
    "git_commit": git_commit,
    "seed": seed,
    "packages": {"pandas": version("pandas"), "numpy": version("numpy")},
}
```

Three things to record for a rerun	Data (content hash or table version), config (hash of the loaded config), code (Git commit).
Hash a large file	with path.open("rb") as f: hashlib.file_digest(f, "sha256") (Python 3.11+).
Stable config hash	sha256 of json.dumps(config, sort_keys=True).encode("utf-8").
Raw inputs	Immutable: each delivery at a new path, never overwritten.

B6-L3 Pinned dependencies and lock files

Ranges in pyproject.toml, exact versions and hashes in a committed lock file, and --locked in CI.

```
uv lock                  # resolve pyproject.toml ranges, write uv.lock
uv sync --locked         # install exactly uv.lock; fail if it is out of date
uv lock --upgrade-package pandas   # move one package on purpose
```

pyproject.toml vs lock file	pyproject.toml: ranges you support. Lock: exact versions and hashes of everything installed.
uv sync --locked	Installs exactly uv.lock and fails if pyproject.toml changed without a new lock.
Hashes without uv	pip-compile --generate-hashes, then pip install --require-hashes -r requirements.txt.

Why not pip freeze?	It captures one environment's accidents, with no hashes and no reasons.